

Integrasi LLM, Kilo Code, dan CoppeliaSim untuk Navigasi serta Pemetaan Robot Pioneer P3DX

Author(s): Muhammad Bari Akmal
Teacher: Muhammad Qomaruz Zaman, S.T., M.T., Ph.D.
Class name: Sistem Robot Otonom
YouTube video link: <https://youtu.be/3LwAsEHLHqU>
GitHub link: <https://github.com/Drimal-835/Otomatic-System>

1 Pendahuluan

Pada proyek ini dibuat sistem kontrol robot bergerak berbasis simulasi menggunakan CoppeliaSim dan model robot Pioneer P3DX. Tujuan utama proyek adalah membuat robot dapat menerima perintah berbasis bahasa natural, menerjemahkan perintah tersebut menjadi misi yang aman, lalu mengeksekusi misi di CoppeliaSim secara *closed-loop*. Sistem ini tidak memberikan kendali roda secara langsung kepada LLM, tetapi menggunakan LLM sebagai penerjemah maksud pengguna menjadi pilihan misi yang sudah didefinisikan pada program Python.

Misi utama yang dikembangkan adalah navigasi menuju target, pemetaan area menggunakan sensor ultrasonik, gerak melingkar, rotasi, gerak maju, dan penghentian robot. Pemetaan area dilakukan dengan pendekatan *wall-following*, yaitu robot mencari dinding, mengikuti kontur dinding, menyimpan titik hasil pembacaan sensor ultrasonik, lalu menghasilkan grafik peta area.

2 Tujuan Proyek

Tujuan dari proyek ini adalah sebagai berikut:

1. Menghubungkan CoppeliaSim dengan Python menggunakan ZeroMQ Remote API.
2. Membuat antarmuka Python satu berkas untuk membaca *state* dan mengirim *action* ke robot P3DX.
3. Membuat sistem *closed-loop* sehingga robot bergerak berdasarkan umpan balik posisi, kecepatan, target, dan sensor ultrasonik.
4. Membuat *agent manager* yang dapat memilih misi berdasarkan perintah pengguna.
5. Mengintegrasikan LLM melalui Kilo Code sehingga pengguna dapat memberi perintah seperti `coppeliassim: mapping the area` atau `coppeliassim: go to the target`.
6. Menghasilkan peta hasil eksplorasi berbasis pembacaan ultrasonik.

3 Arsitektur Sistem

Arsitektur sistem dibagi menjadi beberapa lapisan. Lapisan paling bawah adalah CoppeliaSim yang menjalankan simulasi P3DX. Di atasnya terdapat `p3dx_interface.py` sebagai antarmuka antara Python dan CoppeliaSim. Lapisan berikutnya adalah `p3dx_missions.py`, yang berisi algoritma misi robot. Selanjutnya, `agent_manager.py` bertugas mengubah perintah pengguna menjadi misi tertentu. Pada lapisan paling atas, Kilo Code digunakan sebagai tempat pengguna menulis perintah berbasis bahasa natural.

Table 1: Pembagian modul sistem

Modul	Fungsi
<code>p3dx_interface.py</code>	Menghubungkan Python dengan CoppeliaSim, membaca posisi robot, posisi target, kecepatan, dan sensor ultrasonik, serta mengirim aksi ke motor.
<code>p3dx_missions.py</code>	Menjalankan misi robot seperti <i>go to goal</i> , <i>mapping</i> , <i>circle</i> , <i>rotate</i> , <i>forward</i> , dan <i>stop</i> .
<code>agent_manager.py</code>	Menerjemahkan frasa perintah menjadi pemilihan misi.
<code>kilo_robot_runner.py</code>	Pembungkus keamanan agar Kilo hanya menjalankan misi melalui format perintah yang diizinkan.
Kilo Code	Antarmuka pengguna di VS Code untuk menulis perintah natural dengan awalan <code>coppeliasim:</code> .
LLM	Bertindak sebagai penerjemah maksud perintah pengguna, bukan sebagai pengendali motor langsung.

Alur sistem dapat diringkas sebagai berikut:

Prompt pengguna $\rightarrow$ Kilo Code $\rightarrow$ `kilo_robot_runner.py` $\rightarrow$ `agent_manager.py` $\rightarrow$ `p3dx_missions.py` $\rightarrow$ `p3dx_interface.py` $\rightarrow$ CoppeliaSim

4 State dan Action Robot

State robot yang dibaca dari CoppeliaSim meliputi:

1. Posisi robot, yaitu koordinat x , y , z dan orientasi yaw.
2. Posisi target, yaitu objek dummy `/Goal` pada CoppeliaSim.
3. Kecepatan linear dan kecepatan angular robot.
4. Pembacaan sensor ultrasonik sebanyak 16 sensor.

Aksi robot yang digunakan meliputi:

1. *Body twist*, yaitu kecepatan linear v dan kecepatan angular ω .
2. Kecepatan joint roda kiri dan kanan.
3. Preset aksi seperti maju, rotasi, dan berhenti.

Konversi dari *body twist* ke kecepatan roda menggunakan model diferensial sebagai berikut:

$$\omega_L = \frac{v - \omega \frac{L}{2}}{r} \quad (1)$$

$$\omega_R = \frac{v + \omega \frac{L}{2}}{r} \quad (2)$$

Dengan v adalah kecepatan linear robot, ω adalah kecepatan angular robot, L adalah jarak antar roda, dan r adalah jari-jari roda.

5 Konfigurasi Sensor Ultrasonik

Robot Pioneer P3DX memiliki 16 sensor ultrasonik. Urutan sensor yang digunakan pada proyek ini adalah sebagai berikut:

Table 2: Urutan sensor ultrasonik P3DX

Indeks Sensor	Arah Sensor
0	Kiri
1	Diagonal kiri-depan sekitar 45 derajat
2, 3, 4, 5	Area depan robot
6	Diagonal kanan-depan sekitar 45 derajat
7	Kanan
8	Kanan belakang
9	Diagonal kanan-belakang
10, 11, 12, 13	Area belakang robot
14	Diagonal kiri-belakang
15	Kiri belakang

Pembagian sensor ini penting karena algoritma *wall-following* bergantung pada sensor samping dan sensor diagonal depan. Jika urutan sensor salah, robot dapat berputar terlalu jauh, menjauh dari dinding, atau tidak mengikuti bentuk dinding dengan baik.

6 Integrasi LLM dan Kilo Code

Pada proyek ini, LLM tidak digunakan untuk mengontrol motor secara langsung. LLM hanya digunakan sebagai penerjemah bahasa natural menjadi misi yang aman. Dengan pendekatan ini, perintah seperti:

```
coppeliiasim: go to the target
coppeliiasim: mapping the area
coppeliiasim: make a counter clockwise circle with 0.5 meter diameter
```

akan diteruskan ke program Python melalui `kilo_robot_runner.py`. Awalan `coppeliiasim:` digunakan sebagai pemisah antara percakapan biasa dan perintah untuk mengontrol robot.

Aturan utama pada Kilo Code adalah sebagai berikut:

```
Only control the robot if the user message starts with coppeliiasim:
For robot control, only run:
python kilo_robot_runner.py "coppeliiasim: <user request>"
Do not directly send wheel velocity or motor commands.
```

Dengan aturan ini, sistem menjadi lebih aman karena LLM tidak bebas menjalankan perintah terminal yang tidak diperlukan dan tidak langsung memberikan nilai kecepatan roda.

7 Algoritma Navigasi ke Target

Misi *go to goal* bertujuan membuat robot bergerak menuju objek target `/Goal`. Algoritma ini menggunakan dua mode utama, yaitu:

1. **NORMAL**: robot menghadap ke arah target dan bergerak maju jika area depan aman.
2. **AVOID**: robot mengikuti dinding atau obstacle ketika jalur menuju target terhalang.

Mode *AVOID* menggunakan pendekatan sederhana seperti algoritma Bug. Ketika obstacle terdeteksi, robot memilih sisi dinding yang akan diikuti. Robot akan kembali ke mode *NORMAL* jika jalur ke target kembali terbuka, arah target berada di depan robot, dan jarak robot ke target sudah lebih dekat dibanding saat pertama kali obstacle terdeteksi.

8 Algoritma Pemetaan Area

Misi pemetaan menggunakan sensor ultrasonik untuk memperkirakan posisi dinding di sekitar robot. Setiap pembacaan sensor dikonversi menjadi titik obstacle pada koordinat dunia dengan persamaan:

$$x_o = x_r + d \cos(\theta_r + \theta_s) \quad (3)$$

$$y_o = y_r + d \sin(\theta_r + \theta_s) \quad (4)$$

Dengan (x_o, y_o) adalah titik obstacle, (x_r, y_r) adalah posisi robot, d adalah jarak hasil sensor ultrasonik, θ_r adalah orientasi robot, dan θ_s adalah sudut lokal sensor.

Strategi pemetaan yang digunakan adalah:

1. Robot mencari dinding terdekat.
2. Setelah dinding ditemukan, robot masuk ke mode *FOLLOW_WALL*.
3. Robot mengikuti dinding sambil menyimpan titik hasil sensor ultrasonik.
4. Jika robot kembali mendekati titik awal mengikuti dinding, loop dianggap selesai.
5. Robot bergerak menjauh dari dinding yang telah selesai diikuti untuk mencari dinding lain yang tidak terhubung.

9 Kontrol Wall Following

Masalah utama pada hasil awal adalah robot tidak benar-benar mengikuti bentuk dinding. Jalur robot cenderung membentuk gelombang besar dan tidak sejajar dengan dinding. Hal ini disebabkan oleh kontrol wall following yang masih terlalu reaktif dan penggunaan sensor samping yang terlalu bercampur dengan sensor diagonal atau sensor belakang.

Perbaikan dilakukan dengan menggunakan kontrol proporsional terhadap jarak dinding:

$$e = d_{ref} - d_{side} \quad (5)$$

$$\omega = K_p e + \omega_{corner} \quad (6)$$

Dengan d_{ref} adalah jarak referensi ke dinding, d_{side} adalah jarak sensor samping ke dinding, K_p adalah gain proporsional, dan ω_{corner} adalah koreksi ketika sensor diagonal depan mendeteksi sudut atau belokan.

Untuk mengikuti dinding kanan, sensor utama yang digunakan adalah sensor kanan dan diagonal kanan-depan. Untuk mengikuti dinding kiri, sensor utama yang digunakan adalah sensor kiri dan diagonal kiri-depan. Dengan demikian, robot tidak terlalu sering mengubah arah akibat pembacaan sensor belakang.

10 Hasil Simulasi dan Pemetaan

Gambar berikut menunjukkan hasil pemetaan yang telah dihasilkan oleh robot. Titik-titik pada grafik merupakan hasil pembacaan sensor ultrasonik, sedangkan garis biru menunjukkan lintasan pusat robot selama melakukan eksplorasi.

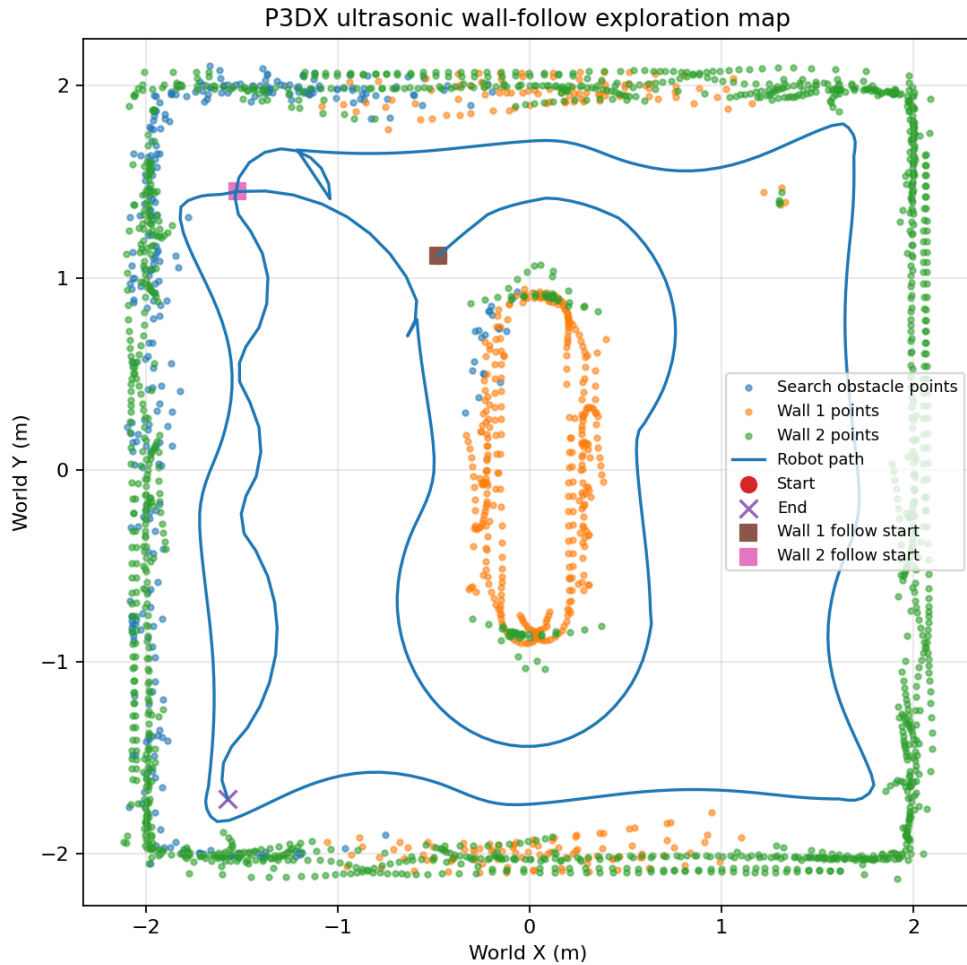


Figure 1: Hasil pemetaan area menggunakan sensor ultrasonik.

Pada hasil pemetaan terlihat bahwa dinding luar dapat terdeteksi dengan cukup jelas oleh sensor ultrasonik. Selain itu, terdapat obstacle atau dinding di bagian tengah arena yang juga terdeteksi, walaupun jumlahnya lebih sedikit dibanding dinding luar. Hal ini menunjukkan bahwa robot lebih banyak menghabiskan waktu mengikuti dinding luar dibanding mengeksplorasi obstacle tengah.

Jika tersedia hasil pemetaan terbaru, gambar berikut dapat digunakan sebagai perbandingan:

File map_graph_final.png belum tersedia.

Figure 2: Hasil pemetaan setelah proses penalaan algoritma wall following.

11 Analisis Hasil

Berdasarkan hasil pemetaan, sistem sudah mampu membaca data ultrasonik, menyimpan titik obstacle, dan menggambarkan peta area dalam bentuk grafik. Dinding luar arena dapat terbentuk dengan cukup baik pada grafik. Namun, lintasan robot masih belum selalu sejajar dengan bentuk dinding. Jalur robot pada beberapa bagian masih membentuk gelombang besar, sehingga dapat disimpulkan bahwa algoritma wall following masih membutuhkan penalaan lebih lanjut.

Beberapa penyebab utama dari masalah tersebut adalah:

1. Kontrol wall following masih menggunakan aturan sederhana sehingga robot mudah beresilasi.
2. Sensor ultrasonik memiliki area deteksi yang lebar, sehingga titik obstacle memiliki sebaran yang cukup besar.
3. Robot belum menggunakan metode SLAM penuh, sehingga belum ada koreksi peta berdasarkan posisi yang lebih akurat.

4. Dinding luar yang besar dapat membuat robot terlalu lama mengikuti dinding yang sama, sehingga obstacle tengah kurang dieksplorasi.

Untuk meningkatkan performa, beberapa perbaikan yang dapat dilakukan adalah:

1. Menggunakan kontrol proporsional-derivatif agar robot lebih stabil saat mengikuti dinding.
2. Menggunakan sensor samping murni untuk menjaga jarak dinding dan sensor diagonal depan untuk mendeteksi sudut.
3. Membuat logika pencarian obstacle tengah setelah loop dinding luar selesai.
4. Menggunakan grid occupancy map agar peta lebih mudah dianalisis.
5. Menambahkan LiDAR virtual jika diizinkan, atau menambahkan sensor proximity tambahan untuk meningkatkan resolusi pemetaan.

12 Pengujian Sistem dengan Kilo Code

Pengujian dilakukan dengan menuliskan perintah melalui Kilo Code pada VS Code. Contoh perintah yang digunakan adalah:

```
coppeliiasim: go to the target
coppeliiasim: mapping the area
coppeliiasim: make a counter clockwise circle with 0.5 meter diameter
```

Ketika perintah diawali dengan `coppeliiasim:`, Kilo menjalankan `kilo_robot_runner.py`. Program tersebut kemudian menjalankan `agent_manager.py` untuk memilih misi yang sesuai. Setelah misi selesai, khusus untuk misi mapping, program membuka file `map_graph.png` sebagai hasil pemetaan.

13 Kesimpulan

Proyek ini berhasil membuat sistem kontrol robot P3DX berbasis CoppeliaSim yang dapat menerima perintah bahasa natural melalui Kilo Code dan menerjemahkannya menjadi misi robot. Sistem telah mendukung navigasi ke target, obstacle avoidance, pemetaan area menggunakan ultrasonik, dan visualisasi hasil mapping dalam bentuk grafik.

Pendekatan LLM sebagai penerjemah misi terbukti lebih aman dibanding memberikan kendali motor langsung kepada LLM. Dengan arsitektur ini, LLM hanya memilih misi yang tersedia, sedangkan eksekusi gerak tetap dikendalikan oleh algoritma Python yang deterministik. Hasil pemetaan menunjukkan bahwa robot sudah mampu mendeteksi dinding luar dan sebagian obstacle tengah, tetapi algoritma wall following masih perlu ditingkatkan agar lintasan robot lebih mengikuti bentuk dinding secara konsisten.

14 Lampiran: Contoh Perintah

```
python agent_manager.py "go to the target"
python agent_manager.py "mapping the area"
python kilo_robot_runner.py "coppeliiasim: mapping the area"
python p3dx_interface.py get-state
python p3dx_interface.py apply-action '{"type":"stop"}'
```
